PROYECTO: Programa de FTP Manual MASTERTECH

Índice

Índice 2
1. Introducción 5
    1.1. Propósito del Sistema 5
    1.2. Audiencia 5
2. Descripción General del Sistema 6
    2.1. Características Principales 6
3. Requisitos del Sistema 7
    3.1. Requisitos del Cliente 7
    3.2. Requisitos del Servidor 7
    3.3. Requisitos de Red 7
4. Arquitectura del Sistema 8
    4.1. Diagrama de Arquitectura 8
    4.2. Flujo de Operación 9
5. Componentes del Sistema 10
    5.1. FTP.py - Interfaz Gráfica 10
        5.1.1. Elementos de la Interfaz 10
        5.1.2. Configuración 10
    5.2. Programa_ftp.sh - Script del Servidor 10
        5.2.1. Estructura 10
        5.2.2. Acciones Disponibles 11
6. Instalación y Configuración 12
    6.1. Configuración del Servidor 12
        Paso 1: Copiar el script al servidor 12
        Paso 2: Dar permisos de ejecución 12
        Paso 3: Configurar sudo sin contraseña 12
    6.2. Configuración del Cliente 12
        Paso 1: Verificar Python 12
        Paso 2: Configurar SSH 12
        Paso 3: Configurar FTP.py 13
        Paso 4: Dar permisos de ejecución 13
    6.3. Verificación de la Instalación 13
7. Guía de Uso 14
    7.1. Iniciar la Aplicación 14
    7.2. Descripción de la Interfaz 14
    7.3. Operaciones Básicas 14
        7.3.1. Crear un Usuario 14

7.3.2. Crear una Carpeta 14
7.3.3. Cambiar Permisos 14
7.3.4. Cambiar Propietario 14
7.3.5. Listar Contenido 15
8. Funciones Disponibles 16
8.1. Gestión de Usuarios 16
Crear Usuario 16
Borrar Usuario 16
8.2. Gestión de Carpetas 16
Crear Carpeta 16
Borrar Carpeta 16
8.3. Gestión de Permisos 16
Cambiar Permisos 16
Cambiar Propietario 17
Listar 17
9. Solución de Problemas 18
9.1. Errores Comunes 18
Error: No se puede conectar al servidor 18
Error: Permission denied 18
Error: Debes introducir un usuario/ruta 18
Error: no se recibio accion 18
9.2. Diagnóstico 18
Verificar conexión SSH 18
Probar el script manualmente 18
Verificar logs del sistema 18
9.3. Contacto de Soporte 18
10. Seguridad 20
10.1. Recomendaciones de Seguridad 20
Autenticación SSH 20
Privilegios del Sistema 20
Gestión de Usuarios 20
10.2. Auditoría y Registro 20
10.3. Copias de Seguridad 20
11. Mantenimiento 21
11.1. Tareas de Mantenimiento Regular 21
Diarias 21
Semanales 21
Mensuales 21

11.2. Actualización del Sistema 21
Actualizar el Cliente 21
Actualizar el Servidor 21
11.3. Limpieza de Sistema 21
12. Código Fuente Detallado 23
12.1. FTP.py - Estructura y Funciones 23
Clase FTPAdmin 23
Método __init__ 23
Método crear_interfaz 23
Método ejecutar 23
Métodos de Acción 23
12.2. Programa_ftp.sh - Estructura 23
Sección de Argumentos 23
Estructura case 24
Validación de Entrada 24
12.3. Flujo de Datos 24

1. Introducción

Este manual describe el funcionamiento del Sistema de Administración FTP, una herramienta de gestión remota que permite administrar usuarios, carpetas y permisos en un servidor FTP mediante una interfaz gráfica intuitiva.

1.1. Propósito del Sistema

El sistema está diseñado para simplificar las tareas de administración de servidores FTP, eliminando la necesidad de acceder manualmente al servidor mediante SSH o terminal. Proporciona una interfaz gráfica amigable que permite:

- Crear y eliminar usuarios del sistema
- Gestionar carpetas y directorios
- Modificar permisos de archivos y directorios
- Cambiar propietarios de archivos
- Listar contenidos de directorios

1.2. Audiencia

Este manual está dirigido a administradores de sistemas, personal técnico y usuarios con conocimientos básicos de sistemas Linux que necesiten gestionar servidores FTP de forma remota.

2. Descripción General del Sistema

El Sistema de Administración FTP es una aplicación cliente-servidor que consta de dos componentes principales:

- Interfaz Gráfica (Cliente): Aplicación Python con Tkinter que se ejecuta en el equipo del administrador
- Script de Servidor: Script Bash que se ejecuta en el servidor FTP y realiza las operaciones solicitadas

2.1. Características Principales

- Interfaz gráfica intuitiva desarrollada en Python/Tkinter
- Comunicación segura mediante SSH
- Ejecución remota de comandos
- Visualización en tiempo real de resultados
- Gestión completa de usuarios y permisos
- Sin necesidad de conocimientos avanzados de terminal

3. Requisitos del Sistema

3.1. Requisitos del Cliente

Sistema operativo:
- Linux (Ubuntu, Debian, CentOS, etc.)
- Windows 10/11 (con Python instalado)
- macOS 10.14 o superior

Software requerido:
- Python 3.6 o superior
- Biblioteca Tkinter (incluida por defecto en Python)
- Cliente SSH (ssh) configurado

3.2. Requisitos del Servidor

Sistema operativo:
- Linux Ubuntu 20.04 LTS o superior
- Debian 10 o superior

Software requerido:
- Servidor SSH (OpenSSH)
- Bash 4.0 o superior
- Privilegios sudo para el usuario de conexión

3.3. Requisitos de Red

- Conectividad de red entre cliente y servidor
- Puerto SSH (22) accesible
- Autenticación SSH configurada (contraseña o clave pública)

4. Arquitectura del Sistema

El sistema utiliza una arquitectura cliente-servidor con los siguientes componentes:

4.1. Diagrama de Arquitectura

```
┌──────────────────────────────────────────────────────────┐
│ CLIENTE (Equipo Local)                                   │
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │ FTP.py                                           │    │
│  │ (Interfaz Tkinter)                               │    │
│  └──────────────────────────────────────────────────┘    │
│                  │                                       │
│                  ▼                                       │
│  ┌──────────────────────────────────────────────────┐    │
│  │ subprocess.run()                                 │    │
│  │ (SSH Client)                                     │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
                  │
                  │ SSH
                  │ (Puerto 22)
                  ▼
┌──────────────────────────────────────────────────────────┐
│ SERVIDOR (192.168.10.10)                                 │
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │ SSH Server                                       │    │
│  └──────────────────────────────────────────────────┘    │
│                  │                                       │
│                  ▼                                       │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Programa_ftp.sh                                  │    │
│  │ (Script Bash)                                    │    │
│  └──────────────────────────────────────────────────┘    │
│                  │                                       │
│                  ▼                                       │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Comandos del Sistema                             │    │
│  │ (adduser, mkdir, etc)                            │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

4.2. Flujo de Operación

1. El usuario interactúa con la interfaz gráfica (FTP.py)
2. La aplicación construye un comando SSH con los parámetros necesarios
3. El comando se envía al servidor mediante SSH
4. El servidor ejecuta el script Programa_ftp.sh con los parámetros recibidos
5. El script realiza la operación solicitada
6. Los resultados se devuelven al cliente
7. La interfaz muestra los resultados en el panel de salida

5. Componentes del Sistema

5.1. FTP.py - Interfaz Gráfica

Archivo principal que contiene la interfaz gráfica de usuario. Desarrollado en Python utilizando la biblioteca Tkinter.

5.1.1. Elementos de la Interfaz

Campos de entrada:

- Usuario: Campo para introducir el nombre del usuario
- Ruta: Campo para especificar la ruta de carpetas o archivos
- Permisos: Campo para definir permisos (valor por defecto: 755)

Secciones funcionales:

- Usuarios: Botones para crear y borrar usuarios
- Carpetas: Botones para crear y borrar carpetas
- Permisos: Botones para cambiar permisos, cambiar propietario y listar contenido
- Salida: Panel de texto con scroll para mostrar resultados de operaciones

5.1.2. Configuración

Variables de configuración al inicio del archivo:

```
SERVER = "infra@192.168.10.10"
```

- Usuario y dirección IP del servidor

```
SCRIPT = "/home/infra/Programa_ftp.sh"
```

- Ruta absoluta del script en el servidor

5.2. Programa_ftp.sh - Script del Servidor

Script Bash que se ejecuta en el servidor y procesa las solicitudes del cliente, ejecutando los comandos del sistema correspondientes.

5.2.1. Estructura

El script recibe cuatro argumentos:

8. ACTION: Acción a realizar
9. USER: Nombre de usuario (opcional según la acción)
10. PATH_ARG: Ruta del archivo o directorio (opcional)
11. PERM: Permisos a aplicar (opcional)

5.2.2. Acciones Disponibles

Acción	Descripción
`creador_usuarios`	Crea un nuevo usuario en el sistema sin contraseña
`borrar_usuarios`	Elimina un usuario y su directorio home
`crear_carpetas`	Crea un directorio incluyendo subdirectorios intermedios
`borrar_carpetas`	Elimina un directorio y todo su contenido recursivamente
`dar_permisos`	Cambia los permisos de un archivo o directorio
`cambiar_propietario`	Cambia el propietario y grupo de un archivo o directorio
`listar`	Lista el contenido de un directorio con detalles

6. Instalación y Configuración

6.1. Configuración del Servidor

Paso 1: Copiar el script al servidor

```
scp Programa_ftp.sh infra@192.168.10.10:/home/infra/
```

Paso 2: Dar permisos de ejecución

```
ssh infra@192.168.10.10
chmod +x /home/infra/Programa_ftp.sh
```

Paso 3: Configurar sudo sin contraseña

Para permitir que el script ejecute comandos con privilegios administrativos sin solicitar contraseña, editar el archivo sudoers:

```
sudo visudo
```

Añadir al final del archivo:

```
infra ALL=(ALL) NOPASSWD: /usr/sbin/adduser, /usr/sbin/deluser, /usr/bin/chown
```

6.2. Configuración del Cliente

Paso 1: Verificar Python

```
python3 --version
```

Debe mostrar Python 3.6 o superior

Paso 2: Configurar SSH

Configurar autenticación SSH mediante clave pública para evitar introducir contraseña en cada operación:

```
# Generar clave SSH (si no existe)
ssh-keygen -t rsa -b 4096
```

```
# Copiar clave al servidor
ssh-copy-id infra@192.168.10.10
```

```
# Verificar conexión sin contraseña
ssh infra@192.168.10.10 'echo Conexión exitosa'
```

Paso 3: Configurar FTP.py

Editar el archivo FTP.py y ajustar las variables de configuración:

```
SERVER = "usuario@IP_SERVIDOR"
SCRIPT = "/ruta/completa/al/script/Programa_ftp.sh"
```

Paso 4: Dar permisos de ejecución

```
chmod +x FTP.py
```

6.3. Verificación de la Instalación

Para verificar que todo está correctamente instalado, ejecutar una prueba:

12. Ejecutar la aplicación: ./FTP.py o python3 FTP.py
13. Introducir una ruta de prueba (ejemplo: /tmp/test)
14. Hacer clic en 'Crear Carpeta'
15. Verificar que aparece el mensaje 'Carpeta creada' en el panel de salida

7. Guía de Uso

7.1. Iniciar la Aplicación

Desde terminal:

```
./FTP.py
```

o bien:

```
python3 FTP.py
```

7.2. Descripción de la Interfaz

La ventana principal de la aplicación está dividida en las siguientes secciones:

- Campos de entrada (Usuario, Ruta, Permisos)
- Sección Usuarios (Crear Usuario, Borrar Usuario)
- Sección Carpetas (Crear Carpeta, Borrar Carpeta)
- Sección Permisos (Cambiar Permisos, Cambiar Propietario, Listar)
- Panel de Salida (muestra resultados de las operaciones)
- Botón Limpiar (borra el contenido del panel de salida)

7.3. Operaciones Básicas

7.3.1. Crear un Usuario

16. Introducir el nombre del usuario en el campo 'Usuario'
17. Hacer clic en el botón 'Crear Usuario'
18. Verificar el resultado en el panel de salida

Ejemplo: Usuario: ftpuser

7.3.2. Crear una Carpeta

19. Introducir la ruta completa en el campo 'Ruta'
20. Hacer clic en el botón 'Crear Carpeta'
21. Verificar el resultado en el panel de salida

Ejemplo: Ruta: /home/ftpuser/documentos

7.3.3. Cambiar Permisos

22. Introducir la ruta del archivo o carpeta en 'Ruta'
23. Introducir los permisos deseados en 'Permisos' (formato octal: 755, 644, etc.)
24. Hacer clic en 'Cambiar Permisos'
25. Verificar el resultado en el panel de salida

7.3.4. Cambiar Propietario

26. Introducir el nombre del usuario en 'Usuario'

27. Introducir la ruta del archivo o carpeta en 'Ruta'
28. Hacer clic en 'Cambiar Propietario'
29. Verificar el resultado en el panel de salida

7.3.5. Listar Contenido

30. Introducir la ruta del directorio en 'Ruta'
31. Hacer clic en 'Listar'
32. El contenido detallado se mostrará en el panel de salida

8. Funciones Disponibles

8.1. Gestión de Usuarios

Crear Usuario

- Función: Crea un nuevo usuario en el sistema
- Comando ejecutado: sudo adduser --disabled-password --gecos "" USUARIO
- Campos requeridos: Usuario
- Nota: El usuario se crea sin contraseña. Para FTP se debe configurar posteriormente.

Borrar Usuario

- Función: Elimina un usuario y su directorio home
- Comando ejecutado: sudo deluser --remove-home USUARIO
- Campos requeridos: Usuario
- Advertencia: Esta acción elimina permanentemente todos los archivos del usuario

8.2. Gestión de Carpetas

Crear Carpeta

- Función: Crea un directorio y todos los subdirectorios necesarios
- Comando ejecutado: mkdir -p RUTA
- Campos requeridos: Ruta
- Ejemplo: /home/ftpuser/documentos/2024

Borrar Carpeta

- Función: Elimina un directorio y todo su contenido
- Comando ejecutado: rm -rf RUTA
- Campos requeridos: Ruta
- Advertencia: Esta acción es irreversible. Todo el contenido será eliminado.

8.3. Gestión de Permisos

Cambiar Permisos

- Función: Modifica los permisos de lectura, escritura y ejecución
- Comando ejecutado: chmod PERMISOS RUTA
- Campos requeridos: Ruta, Permisos

Valores comunes de permisos:

- 755: Propietario: rwx, Grupo: r-x, Otros: r-x
- 644: Propietario: rw-, Grupo: r--, Otros: r--
- 700: Propietario: rwx, Grupo: ---, Otros: ---

- 777: Todos: rwx (no recomendado por seguridad)

Cambiar Propietario

- Función: Cambia el propietario y grupo de un archivo o directorio
- Comando ejecutado: sudo chown USUARIO:USUARIO RUTA
- Campos requeridos: Usuario, Ruta

Listar

- Función: Muestra el contenido detallado de un directorio
- Comando ejecutado: ls -lah RUTA
- Campos requeridos: Ruta
- Información mostrada: Permisos, propietario, grupo, tamaño, fecha de modificación, nombre

9. Solución de Problemas

9.1. Errores Comunes

Error: No se puede conectar al servidor

Posibles causas y soluciones:

- Causa: El servidor no está accesible
  Solución: Verificar conectividad con ping 192.168.10.10
- Causa: Credenciales SSH incorrectas
  Solución: Verificar usuario y configuración SSH
- Causa: Puerto SSH bloqueado
  Solución: Verificar firewall y reglas de red

Error: Permission denied

Posibles causas y soluciones:

- Causa: El usuario no tiene permisos sudo
  Solución: Configurar sudoers correctamente (ver sección 6.1)
- Causa: El script no tiene permisos de ejecución
  Solución: chmod +x /home/infra/Programa_ftp.sh

Error: Debes introducir un usuario/ruta

- Causa: Campos requeridos vacíos
  Solución: Completar todos los campos necesarios antes de ejecutar la acción

Error: no se recibio accion

- Causa: Problema en la comunicación con el script
  Solución: Verificar que la ruta del script en FTP.py sea correcta

9.2. Diagnóstico

Verificar conexión SSH

```
ssh infra@192.168.10.10 'echo Conexion exitosa'
```

Probar el script manualmente

```
ssh infra@192.168.10.10 '/home/infra/Programa_ftp.sh listar "" "/tmp" ""'
```

Verificar logs del sistema

```
ssh infra@192.168.10.10 'sudo tail -f /var/log/auth.log'
```

9.3. Contacto de Soporte

Si los problemas persisten después de seguir esta guía, contactar con el equipo de soporte técnico proporcionando:

- Descripción detallada del problema
- Mensajes de error completos
- Pasos realizados antes del error
- Versión del sistema operativo

10. Seguridad

10.1. Recomendaciones de Seguridad

Autenticación SSH

- Usar siempre autenticación por clave pública en lugar de contraseña
- Proteger las claves privadas con contraseña
- Limitar el acceso SSH solo a IPs autorizadas
- Cambiar el puerto SSH por defecto (22) a uno personalizado

Privilegios del Sistema

- Limitar permisos sudo solo a los comandos necesarios
- Revisar periódicamente el archivo /etc/sudoers
- Auditar regularmente las acciones realizadas
- No usar el usuario root directamente

Gestión de Usuarios

- Establecer contraseñas fuertes para usuarios FTP
- Implementar políticas de caducidad de contraseñas
- Eliminar usuarios inactivos regularmente
- Aplicar el principio de mínimos privilegios

10.2. Auditoría y Registro

El sistema registra todas las operaciones en los logs del sistema. Se recomienda:

- Revisar regularmente /var/log/auth.log
- Implementar un sistema de alertas para actividades sospechosas
- Mantener backups de los logs
- Documentar todos los accesos administrativos

10.3. Copias de Seguridad

Es fundamental mantener copias de seguridad de:

- Archivos del script (FTP.py y Programa_ftp.sh)
- Configuración SSH
- Lista de usuarios del sistema
- Datos importantes de usuarios FTP

11. Mantenimiento

11.1. Tareas de Mantenimiento Regular

Diarias

- Verificar que el servicio SSH esté funcionando
- Revisar logs de acceso

Semanales

- Auditar usuarios activos
- Revisar uso de espacio en disco
- Verificar permisos de directorios críticos

Mensuales

- Actualizar el sistema operativo
- Revisar y actualizar políticas de seguridad
- Verificar integridad de copias de seguridad
- Limpiar usuarios y carpetas obsoletas

11.2. Actualización del Sistema

Actualizar el Cliente

33. Realizar copia de seguridad del archivo FTP.py actual
34. Descargar la nueva versión
35. Actualizar las variables de configuración (SERVER, SCRIPT)
36. Probar la nueva versión

Actualizar el Servidor

37. Realizar copia de seguridad del script actual
38. Copiar el nuevo script al servidor
39. Dar permisos de ejecución
40. Verificar funcionamiento

11.3. Limpieza de Sistema

Comandos útiles para limpieza:

```
# Eliminar archivos temporales
sudo rm -rf /tmp/*
```

```
# Limpiar logs antiguos
sudo journalctl --vacuum-time=30d
```

```
# Verificar espacio en disco
df -h
```

12. Código Fuente Detallado

12.1. FTP.py - Estructura y Funciones

Clase FTPAdmin

La clase principal que gestiona la interfaz y las operaciones del sistema.

Método __init__

Constructor que inicializa la ventana principal:

- Configura el título de la ventana
- Define el tamaño (700x550 píxeles)
- Llama a crear_interfaz() para construir la GUI

Método crear_interfaz

Construye todos los elementos visuales:

- Frame principal con padding
- Labels y Entry para campos de entrada
- LabelFrame para agrupar botones por categoría
- ScrolledText para el panel de salida
- Botones con comandos asociados

Método ejecutar

Método central que gestiona la comunicación con el servidor:

41. Valida que se haya especificado una acción
42. Construye el comando SSH con los parámetros
43. Ejecuta el comando usando subprocess.run()
44. Captura stdout y stderr
45. Muestra los resultados en el panel de salida
46. Maneja excepciones y muestra errores

Métodos de Acción

Cada botón tiene un método asociado que:

- Obtiene los valores de los campos de entrada
- Valida que se hayan proporcionado los datos necesarios
- Llama al método ejecutar() con los parámetros correctos

12.2. Programa_ftp.sh - Estructura

Sección de Argumentos

El script captura los argumentos posicionales:

```
ACTION="$1"  # Acción a realizar
```

```
USER="$2"    # Usuario (opcional)
PATH_ARG="$3" # Ruta (opcional)
PERM="$4"    # Permisos (opcional)
```

Estructura case

Utiliza una estructura case para evaluar la acción y ejecutar el comando correspondiente. Cada caso:

- Valida los parámetros necesarios
- Ejecuta el comando del sistema
- Muestra un mensaje de confirmación
- Retorna código de salida apropiado

Validación de Entrada

Cada acción verifica que los parámetros requeridos estén presentes antes de ejecutar el comando, evitando errores de ejecución.

12.3. Flujo de Datos

Ejemplo completo: Crear un usuario llamado 'nuevo_usuario'

47. Usuario introduce 'nuevo_usuario' en el campo Usuario
48. Hace clic en 'Crear Usuario'
49. Se ejecuta: crear_usuario()
50. Se llama a: ejecutar('creador_usuarios', 'nuevo_usuario')
51. Se construye el comando SSH
52. El servidor ejecuta Programa_ftp.sh con los parámetros
53. El script ejecuta: sudo adduser --disabled-password nuevo_usuario
54. Se devuelve: 'Usuario nuevo_usuario creado'
55. El mensaje se muestra en el panel de salida